

Exercices 16 — Tri par sélection

Semaine 16 ■ À faire sans machine ■ 1 h

Exercice 1 — Dérouler le tri ★

Trier [6, 2, 9, 4, 1] par sélection : donner l'état du tableau après **chaque échange** (souligner la zone triée).

Exercice 2 — Questions de coût ★

1. Combien de comparaisons pour trier 100 éléments par sélection ?
2. Le tableau est déjà trié : combien de comparaisons ?
3. Un tri de 10 000 éléments prend 2 secondes. Estimer la durée pour 100 000 éléments.

Exercice 3 — Le bug du None ★★

```
notes = [12, 9, 15]
notes = tri_selection(notes)
print(notes[0])
```

Qu'affiche ce programme ? Expliquer et corriger.

Exercice 4 — Lire l'invariant ★★

Pendant le tri par sélection de [3, 1, 4, 1, 5, 9, 2, 6], on photographie le tableau : [1, 1, 2, 4, 5, 9, 3, 6].

1. Quelle est la valeur de *i* (début d'étape) compatible avec cette photo ? Justifier avec l'invariant.
2. La photo [1, 2, 1, 4, 5, 9, 3, 6] est-elle possible ?

Exercice 5 — Variantes d'écriture ★★

1. Réécrire `tri_selection` pour trier par ordre **décroissant**.
2. Écrire `tri_selection_copie(t)` qui renvoie un **nouveau** tableau trié sans modifier `t` (deux lignes, en réutilisant `tri_selection`).

Exercice 6 — Tester un tri ★★★

On veut un jeu de tests pour `tri_selection`.

1. Pourquoi `assert tri_selection([3, 1]) == [1, 3]` est-il doublement faux ?
2. Écrire trois asserts corrects (cas normal, déjà trié, doublons).
3. ★★★ Un tri bugué pourrait renvoyer [1, 1] pour [3, 1] : trié, mais faux ! Quelles sont les **deux** propriétés qu'un résultat de tri doit vérifier ?